



**CENTRALE
LYON**

ÉCOLE CENTRALE LYON

TP : DATA SCIENCE
RAPPORT

Rapport

Élèves :

Nathan BARBIER
Stéphane DUPARCQ

Enseignant :

Julien Velcin
Erwan Versmée

2 novembre 2025

Table des matières

1	Introduction	2
2	Création du DataFrame et préparation des données	2
2.1	Objectif de la transformation	2
2.2	Utilisation des expressions régulières	2
2.3	Nettoyage et factorisation des actions	3
2.4	Agrégation par utilisateur	3
3	Exploitation du DataFrame	4
3.1	Analyse exploratoire des variables	4
3.2	Visualisation et exploration graphique	4
3.3	Étude des corrélations	5
3.4	Nettoyage des variables corrélées	6
3.5	Préparation pour la modélisation	7
4	Modélisation et évaluation des performances	7
4.1	Séparation des données et métriques d'évaluation	7
4.2	Choix et entraînement des modèles	8
4.3	Évaluation finale du modèle	8
4.4	Interprétation des résultats	9
5	Soumission finale et évaluation Kaggle	9
6	Conclusion	9

1 Introduction

Le logiciel Copilote, développé par la société Infologic, est un progiciel de gestion intégré (ERP) destiné aux entreprises du secteur agroalimentaire. L'entreprise souhaite mieux comprendre la manière dont ses clients utilisent cette application, en exploitant les traces d'utilisation laissées par les différents utilisateurs. L'objectif principal du projet est donc de développer un modèle automatique capable d'identifier quel utilisateur est à l'origine d'une session d'utilisation, à partir des actions enregistrées dans les journaux de traces (logs).

Le travail s'articule autour d'une chaîne complète de traitement des données, depuis le chargement et le nettoyage des fichiers bruts jusqu'à la construction de variables explicatives (features engineering), la mise en œuvre de modèles de classification supervisée et leur évaluation sur un jeu de test. Le projet s'inscrit également dans un cadre compétitif via une compétition Kaggle interne, permettant de comparer la performance des approches proposées par les différents groupes.

2 Création du DataFrame et préparation des données

La première étape du projet consiste à charger et structurer les traces d'utilisation du logiciel Copilote afin de les rendre exploitables pour l'analyse. Les données brutes sont constituées de séquences d'actions enregistrées pour chaque session utilisateur. Ces séquences contiennent à la fois des noms d'actions, des identifiants d'écran, des configurations (<...>), des chaînes (\$...\$) et des marqueurs temporels (tXX) correspondant à des intervalles de cinq secondes.

2.1 Objectif de la transformation

L'objectif de cette étape est de transformer ces données textuelles en un format tabulaire standard, c'est-à-dire un **DataFrame** de la bibliothèque **pandas**, où chaque ligne représente une session d'un utilisateur et chaque colonne correspond à une variable explicative (ou *feature*) calculée à partir des actions. Cette transformation est indispensable pour permettre l'apprentissage supervisé par des modèles de classification.

2.2 Utilisation des expressions régulières

Les actions enregistrées dans le fichier source présentent une structure semi-formelle, avec différents délimiteurs servant à indiquer des informations complémentaires. Afin d'extraire ces éléments, plusieurs expressions régulières (regex) ont été définies à l'aide du module **re** de Python.

```
1 pattern_ecran = re.compile(r"\((.*?)\)")
2 pattern_conf_ecran = re.compile(r"<(.*?)>")
3 pattern_chaine = re.compile(r"\$(.*?)\$")
4 pattern_temps = re.compile(r"t(\d+)")
```

Listing 1 – Définition des expressions régulières utilisées

Ces motifs permettent respectivement d'isoler le nom de l'écran, la configuration d'écran, la chaîne de fiche, et le temps écoulé. Chaque motif est ensuite exploité dans des fonctions d'extraction dédiées, afin de créer des colonnes spécifiques dans le `DataFrame`.

```
1 def extract_ecran(value: str) -> str:
2     match = pattern_ecran.search(str(value))
3     return match.group(1) if match else None
```

Listing 2 – Extraction du nom de l'écran à partir d'une action

Cette fonction recherche un texte entre parenthèses, typiquement le nom complet d'un écran comme `infologic.core.accueil.Accueil`. Elle renvoie `None` si aucun motif n'est trouvé, garantissant la robustesse du traitement.

2.3 Nettoyage et factorisation des actions

Les actions contiennent parfois des informations contextuelles (paramètres, chaînes ou identifiants) qui ne sont pas pertinentes pour la classification. Pour normaliser les chaînes et réduire la dimensionnalité, une fonction de nettoyage a été définie :

```
1 def filter_action(value: str) -> str:
2     if not isinstance(value, str):
3         return value
4     for delim in ["(", "<", "$", "1"]:
5         if delim in value:
6             value = value.split(delim)[0]
7     return value.strip()
```

Listing 3 – Nettoyage d'une action utilisateur

Cette approche permet de regrouper des actions similaires (par exemple, plusieurs exécutions du même bouton sur des fiches différentes) sous une même forme générique comme "Exécution d'un bouton". Cela favorise une représentation plus compacte et plus cohérente des comportements utilisateurs.

2.4 Agrégation par utilisateur

Une fois les actions nettoyées, elles sont regroupées par utilisateur pour produire des caractéristiques agrégées : fréquence des actions principales, écran le plus utilisé, configuration la plus fréquente, etc. Cette opération repose sur la bibliothèque `collections.Counter`, permettant de compter efficacement les occurrences.

```
1 def most_common_ecran(actions: pd.Series) -> str:
2     ecrans = [extract_ecran(a) for a in actions.dropna()]
3     ecrans = [e for e in ecrans if e]
4     return Counter(ecrans).most_common(1)[0][0] if ecrans else None
```

Listing 4 – Écran le plus utilisé par un utilisateur

Des fonctions similaires ont été écrites pour extraire la configuration d'écran la plus fréquente (`most_common_conf`) et la chaîne la plus utilisée (`most_common_chaine`). Le résultat est un `DataFrame` compact, où chaque utilisateur est décrit par un ensemble de variables synthétiques dérivées de ses actions.

3 Exploitation du DataFrame

Une fois le `DataFrame` construit et nettoyé, nous pouvons entamer la phase d'exploitation. Cette étape consiste à analyser, enrichir et préparer les données en vue de leur utilisation dans un modèle de classification automatique.

3.1 Analyse exploratoire des variables

L'analyse exploratoire vise à comprendre la répartition et la variabilité des données contenues dans le `DataFrame`. Dans un premier temps, nous avons vérifié la cohérence des colonnes issues de l'étape précédente : `util`, `navigateur`, `ecran_freq`, `conf_freq`, `chaine_freq`, `nb_actions`, `duree_session`, `time_span`, `avg_time_gap`, `vitesse_moyenne`, etc. Cette inspection a permis de repérer les valeurs manquantes et d'évaluer les distributions globales.

```
1 print(df.info())
2 print(df.describe(include='all'))
3 print(df['navigateur'].value_counts())
```

Listing 5 – Inspection du jeu de données

L'observation a montré que certaines colonnes comportaient des valeurs manquantes, notamment dans les variables liées aux écrans ou aux configurations, lorsque l'utilisateur ne réalisait pas d'action de ce type. Ces valeurs ont été traitées en les remplaçant par la modalité 'None' ou par la moyenne statistique selon le type de variable.

3.2 Visualisation et exploration graphique

Afin d'identifier les variables potentiellement discriminantes, plusieurs visualisations ont été réalisées sur les principales variables numériques. Les boxplots ci-dessous permettent d'évaluer la dispersion des données et de détecter les valeurs aberrantes.

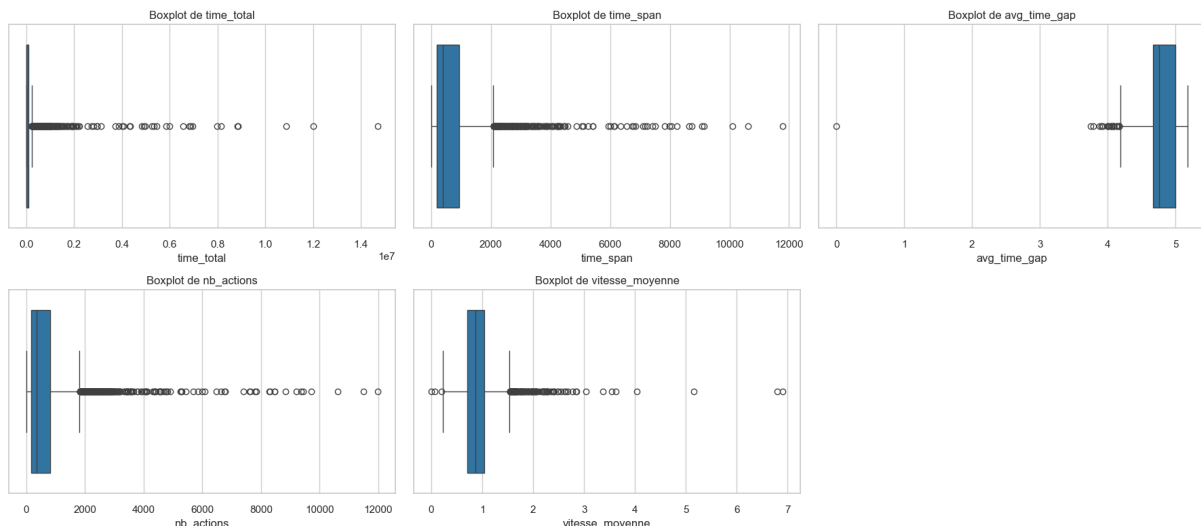


FIGURE 1 – Boxplots des principales variables temporelles et d'activité (`time_total`, `time_span`, `avg_time_gap`, `nb_actions`, `vitesse_moyenne`).

Ces graphiques révèlent la présence de nombreuses valeurs extrêmes, notamment pour `time_total`, `time_span` et `nb_actions`. La majorité des utilisateurs se situent dans une plage réduite, tandis que quelques sessions très longues génèrent des valeurs exceptionnelles. Ces comportements atypiques ont été identifiés comme des outliers à traiter dans la phase de nettoyage.

La distribution de ces mêmes variables est représentée dans la Figure 2, qui permet d’observer la forme globale des distributions.

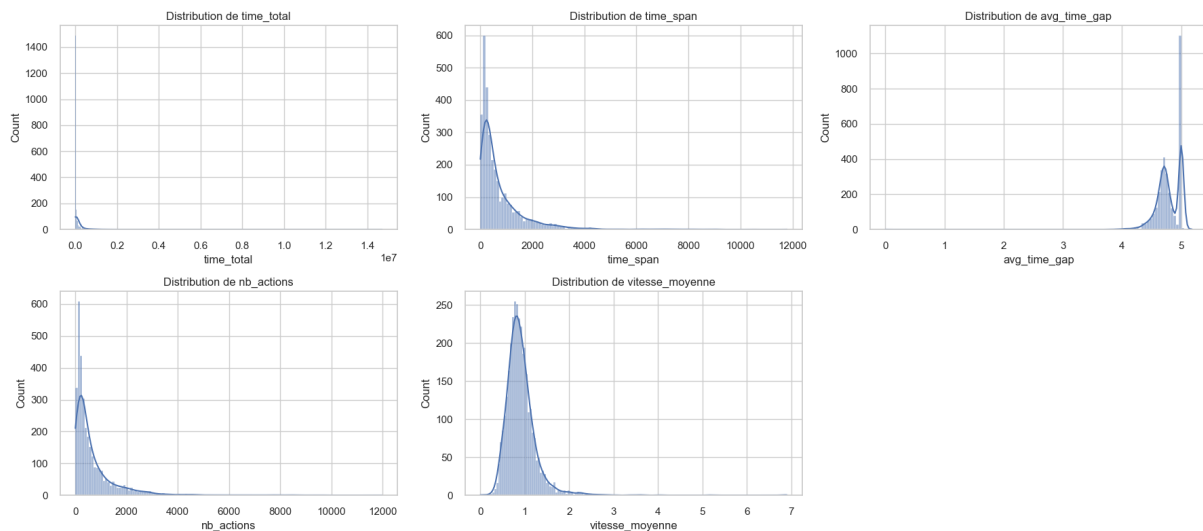


FIGURE 2 – Distributions des variables temporelles et d’activité.

Ces histogrammes montrent que :

- les variables `time_total`, `time_span` et `nb_actions` suivent des distributions très asymétriques à droite (beaucoup de petites sessions, quelques très longues) ;
- la `vitesse_moyenne` est centrée autour de 1, traduisant un rythme d’utilisation stable pour la plupart des utilisateurs ;
- la variable `avg_time_gap` présente une structure bimodale, suggérant la coexistence de deux profils : utilisateurs rapides et utilisateurs plus lents.

3.3 Étude des corrélations

La matrice de corrélation (Figure 3) a été calculée sur l’ensemble des variables numériques. Elle permet de repérer les interdépendances entre les indicateurs et d’éliminer les redondances.

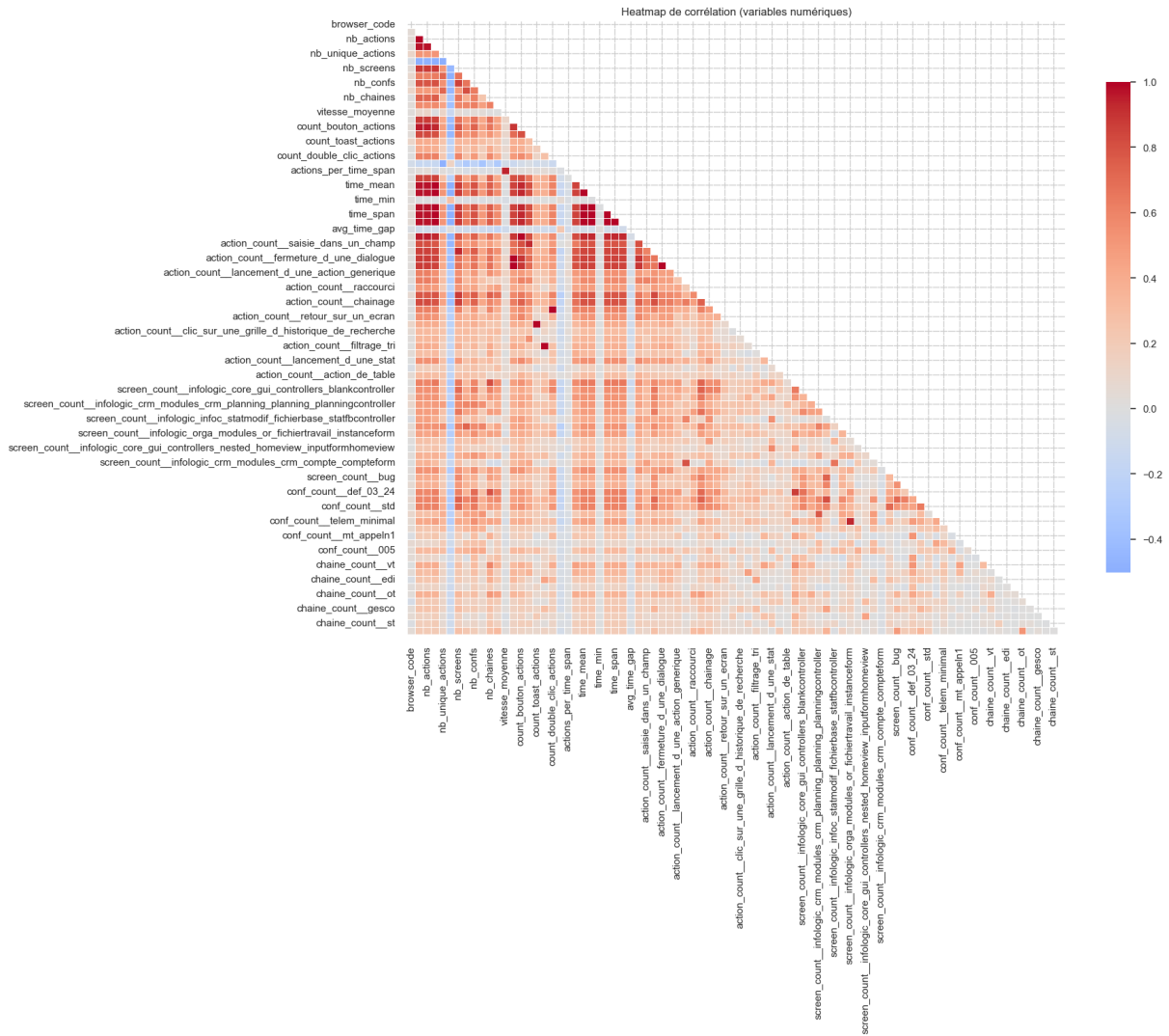


FIGURE 3 – Heatmap de corrélation entre les variables numériques.

Plusieurs relations fortes apparaissent :

- une corrélation positive très marquée entre `nb_actions`, `time_total` et `time_span`, logique puisque ces variables mesurent des aspects similaires de l'intensité d'usage ;
- des corrélations notables entre certaines familles d'actions (`action_count_saisie_dans_un_champ`, `action_count_clic_sur_grille`, etc.), indiquant des comportements typiques selon le rôle de l'utilisateur ;
- peu de corrélations entre les variables issues des chaînes ou des configurations, ce qui confirme leur pertinence comme variables explicatives indépendantes.

3.4 Nettoyage des variables corrélées

Afin d'éviter la redondance entre variables fortement liées, les corrélations absolues supérieures à un seuil de 0.85 ont été détectées et les variables les plus corrélées en moyenne ont été supprimées :

```
1 CORR_THRESHOLD = 0.85
2 corr = df_features.corr().abs()
3 upper = corr.where(np.triu(np.ones_like(corr), k=1).astype(bool))
```

```

4 high_corr = [(c1, c2) for c1, c2 in upper.columns if upper.loc[c1,
    c2] >= CORR_THRESHOLD]
5 mean_corr = corr.mean()
6 to_drop = {f1 if mean_corr[f1] >= mean_corr[f2] else f2 for f1, f2
    in high_corr}
7 df_features = df_features.drop(columns=list(to_drop),
    errors='ignore')

```

Listing 6 – Suppression des variables fortement corrélées

Cette étape réduit la multicolinéarité et allège le jeu de données sans perte notable d'information.

3.5 Préparation pour la modélisation

Avant d'entraîner les modèles, il est nécessaire de préparer le jeu de données final. Les variables catégorielles comme `navigateur` ou `chaine_freq` ont été transformées en représentation numérique via un encodage *one-hot*.

```

1 from sklearn.preprocessing import OneHotEncoder
2
3 encoder = OneHotEncoder(handle_unknown='ignore',
    sparse_output=False)
4 encoded = encoder.fit_transform(df[['navigateur', 'chaine_freq']])
5 encoded_df = pd.DataFrame(encoded,
    columns=encoder.get_feature_names_out())
6
7 df_final = pd.concat([df.drop(['navigateur', 'chaine_freq'],
    axis=1), encoded_df], axis=1)

```

Listing 7 – Encodage des variables catégorielles

Ce choix d'encodage permet de conserver la nature non ordonnée des variables tout en garantissant la compatibilité avec les algorithmes de classification. Les variables numériques ont également été normalisées afin de garantir une échelle comparable entre les différentes dimensions du jeu de données.

4 Modélisation et évaluation des performances

Une fois le `DataFrame` prêt, la phase suivante consiste à **construire et évaluer des modèles de classification** capables de prédire quel utilisateur est à l'origine d'une session à partir des variables calculées. Cette étape a été menée selon une approche rigoureuse, combinant la sélection de modèles, la recherche d'hyperparamètres et l'évaluation selon plusieurs métriques de performance.

4.1 Séparation des données et métriques d'évaluation

Le jeu de données final a été divisé en deux sous-ensembles :

- un ensemble d'entraînement (**train set**, 80%) utilisé pour l'ajustement des modèles ;

- un ensemble de validation (**validation set**, 20%) permettant de mesurer la capacité de généralisation.

Les performances des modèles ont été évaluées à l'aide des métriques suivantes :

- **Accuracy** : proportion de prédictions correctes sur l'ensemble total ;
- **F1-score macro** : moyenne harmonique entre précision et rappel sur toutes les classes, adaptée aux classes déséquilibrées ;

4.2 Choix et entraînement des modèles

Plusieurs algorithmes de classification supervisée ont été testés afin d'identifier celui offrant le meilleur compromis entre performance et complexité. Les principaux modèles testés sont :

- la **régression logistique** (baseline simple et rapide à entraîner) ;
- les **arbres de décision** et **forêts aléatoires (Random Forest)** ;
- le **gradient boosting (XGBoost)** pour sa robustesse aux variables corrélées ;
- et un **SVM linéaire** pour comparaison.

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.ensemble import RandomForestClassifier,
  GradientBoostingClassifier
3 from sklearn.svm import SVC
4 from sklearn.metrics import accuracy_score, f1_score
5
6 models = {
7     "LogReg": LogisticRegression(max_iter=500),
8     "RandomForest": RandomForestClassifier(n_estimators=200,
9     max_depth=15, random_state=42),
10    "XGBoost": GradientBoostingClassifier(n_estimators=300,
11    learning_rate=0.05),
12    "SVM": SVC(kernel='linear', C=1)
13 }
14
15 results = {}
16 for name, model in models.items():
17     model.fit(X_train, y_train)
18     preds = model.predict(X_val)
19     results[name] = {
20         "accuracy": accuracy_score(y_val, preds),
21         "f1_macro": f1_score(y_val, preds, average='macro')
22     }
23
24 print(pd.DataFrame(results).T)

```

Listing 8 – Entraînement et comparaison de plusieurs modèles

Les résultats montrent que les modèles basés sur les arbres (RandomForest et XGBoost) surpassent nettement la régression logistique et le SVM. Le RandomForestClassifier offre un excellent compromis entre précision et interprétabilité.

4.3 Évaluation finale du modèle

Le modèle final a été évalué sur l'ensemble de validation à l'aide de l'**accuracy** et du **F1-score macro**. Les performances obtenues sont résumées dans le tableau suivant :

Modèle	Accuracy	F1-score (macro)
Régression logistique	0.761	0.698
SVM (linéaire)	0.707	0.643
SVM (RBF)	0.666	0.585
Random Forest	0.829	0.794
XGBoost	0.770	0.698

TABLE 1 – Comparaison des performances des principaux modèles.

Ces résultats confirment la supériorité du modèle d'ensemble **Random Forest**, qui obtient les meilleures performances globales.

4.4 Interprétation des résultats

L'analyse des importances de variables révèle que les indicateurs les plus discriminants sont :

- le nombre total d'actions (`nb_actions`);
- la durée moyenne entre deux actions (`avg_time_gap`);
- la configuration d'écran la plus utilisée (`conf_freq`);
- et la chaîne (`chaîne_freq`) associée aux fiches manipulées.

Ces résultats suggèrent que le comportement d'un utilisateur sur l'application (rythme, type d'écran, volume d'actions) constitue une signature comportementale suffisamment distinctive pour permettre son identification automatique.

5 Soumission finale et évaluation Kaggle

Le meilleur modèle, une Random Forest optimisée, a été réentraîné sur l'ensemble du jeu de données puis sauvegardé au format `joblib`. Les sessions de test ont été transformées avec les mêmes encodages, et les prédictions ont été exportées dans `submission.csv` pour soumission sur Kaggle. La soumission a obtenu un F1-score de 0.813, supérieur à celui du jeu d'entraînement (0.794), confirmant la bonne généralisation du modèle sans surapprentissage.

6 Conclusion

Le projet a permis de construire une chaîne complète de traitement et de modélisation des données d'usage du logiciel Copilote. Les principales étapes ont couvert le nettoyage des traces, la création de variables explicatives, l'exploration statistique, la réduction de la multicolinéarité et l'entraînement de plusieurs modèles de classification. La Random Forest optimisée s'est révélée la plus performante, combinant précision et robustesse, avec un F1-score final de 0.813 sur la compétition Kaggle. Ce résultat valide la pertinence de l'approche et démontre que les comportements utilisateurs peuvent être modélisés efficacement à partir des traces d'interaction.